



Universidad de Concepción
Facultad de Ingeniería
Departamento Informática y Ciencias de la Computación
Carrera Ingeniería Civil Informática

Proyecto Semestral: Mapa Turístico

Modelamiento Espacial y Cálculo de Rutas mediante Teoría de Grafos

Curso: Matemáticas Discretas

Profesora: Lilian Angelica Salinas Ayala

Integrantes:

Juan Umaña
Ignacio García
Carlos Amigo

Fecha de Entrega: 11 de mayo de 2026

1. Introducción

El presente informe detalla el desarrollo de una solución computacional para el problema de la navegación urbana y el ruteo turístico. La tarea consiste en transformar una descripción textual de calles y puntos de interés en un modelo matemático funcional que permita determinar rutas transitables para un usuario. El desafío principal radica en la abstracción de elementos geométricos (segmentos de recta) hacia estructuras discretas (grafos), donde la topología de la ciudad dictamina las posibilidades de movimiento.

Nuestra solución propuesta utiliza un enfoque de grafos espaciales, donde se identifican automáticamente los puntos de colisión entre calles para generar una red de tránsito. Para la búsqueda de rutas, se ha implementado un algoritmo de Búsqueda en Anchura (BFS), el cual garantiza encontrar un camino entre dos puntos de interés si este existe. El texto se organiza comenzando por la definición del modelo matemático y geométrico, seguido por las decisiones de arquitectura de software en lenguaje C, para finalizar con los resultados y conclusiones técnicas del proyecto.

2. Objetivos

Objetivo Principal:

Diseñar e implementar un sistema capaz de modelar un mapa urbano a partir de coordenadas reales y calcular rutas de navegación entre múltiples puntos de interés mediante el uso de algoritmos de teoría de grafos.

Objetivos Específicos:

- Implementar un motor de geometría analítica robusto para la detección de intersecciones mediante algoritmos de orientación.
- Modelar una estructura de datos de grafo eficiente que soporte nodos heterogéneos (intersecciones y puntos turísticos).
- Garantizar la integridad y estabilidad del sistema bajo restricciones de memoria estática, optimizando el acceso y la gestión de adyacencias.
- Validar la conectividad del mapa mediante recorridos de grafos para ofrecer instrucciones de navegación claras al usuario.

3. Modelo

El modelo matemático se fundamenta en la representación de la ciudad como un grafo no dirigido $G = (V, E)$. La generación de este modelo se divide en tres fases críticas:

3.1. Definición de Nodos (V)

Se han definido dos tipos de vértices para cubrir la realidad del mapa:

- **Intersecciones:** Puntos donde dos segmentos de calle se cruzan.
- **Puntos Turísticos:** Ubicaciones específicas sobre una calle destinadas a ser visitadas.

3.2. Detección de Intersecciones

Para determinar la existencia de una intersección entre dos calles, se utiliza el Algoritmo de Orientación Geométrica basado en el producto cruz. Este método evita el uso de pendientes, previniendo errores de división por cero en calles verticales. Posteriormente, se utiliza la Regla de Cramer para resolver el sistema de ecuaciones lineales resultante de las rectas y obtener las coordenadas (x, y) exactas del punto de colisión.

3.3. Lógica de Adyacencia (E)

La adyacencia se define por la proximidad consecutiva. Para cada calle, se recolectan todos los nodos que residen en ella y se ordenan linealmente según su distancia euclidiana respecto al origen de la calle. Se establece una arista entre dos nodos si son vecinos inmediatos en esta lista ordenada, garantizando un tránsito realista por los tramos de calle.

4. Implementación

La implementación se realizó en lenguaje C, priorizando la eficiencia y el manejo estático de recursos.

4.1. Estructuras de Datos y Polimorfismo

Se utilizó el patrón *Tagged Union* mediante la estructura *Nodo*, que integra un enum (*TipoNodo*) y una unión para almacenar los datos de una Intersección o de un Punto Turístico sin pérdida de seguridad de tipos.

4.2. Arquitectura del Grafo

La estructura principal *mi_grafo* contiene un arreglo de nodos de tamaño fijo (*MAX NODOS*). Cada nodo gestiona sus adyacencias mediante un arreglo

estático entre vecinos[MAX VECINOS]. Se definió MAX VECINOS = 100, justificándose por el peor caso topológico de convergencia de calles.

4.3. Manejo de Memoria y Algoritmo

Esta arquitectura estática ofrece determinismo y evita fugas de memoria al prescindir de malloc. Para el cálculo de rutas, la función hacer ruta implementa un algoritmo **BFS (Breadth-First Search)**, explorando el grafo nivel por nivel para asegurar la conectividad entre los puntos de interés.

4.4. Estructuras Auxiliares (Cola para BFS)

Para el algoritmo de BFS, fue necesario implementar una estructura para manejar los nodos por visitar. Esta cola se implementó de forma estática mediante un arreglo llamado cola de tamaño MAX_NODOS. Para controlar el flujo de datos en la que se utilizaron dos variables enteras: frente (que indica el índice del elemento a extraer) y final (que indica el índice donde se insertará el próximo elemento). El agregar elementos se maneja aumentando la variable final e insertando el ID del nodo en el arreglo, mientras que para quitar elementos (avanzar en el recorrido), se lee el valor en la posición frente y luego se incrementa dicho índice. Además, se utilizó un arreglo booleano visitado para marcar los nodos ya procesados y evitar ciclos infinitos.

4.5. Consideraciones de compilación

El sistema fue estructurado de manera modular, por lo que para su correcta compilación en Linux o mediante GCC, es necesario enlazar la librería matemática nativa de C, debido al uso de funciones geométricas en el cálculo de las intersecciones. El comando de compilación requerido es:

```
TareaMD\project> gcc main.c archivotxt.c bfs.c city_model.c graph.c -o main -lm ; if ($?) { .\main }
```

5. Conclusiones

Desde una perspectiva técnica, el modelamiento de una ciudad a través de grafos discretos demuestra ser una herramienta eficaz para la resolución de problemas de navegación. La implementación de una estructura estática garantizó robustez frente a errores de memoria comunes en C. El uso de algoritmos geométricos para la autogeneración de nodos dota al sistema de versatilidad para procesar diversos mapas.

Inicialmente, abstraer el modelo matemático del grafo a partir de cruces y coordenadas representó un desafío significativo. Sin embargo, tras un periodo de análisis y discusión grupal, se logró consolidar una abstracción lógica sólida, lo que facilitó enormemente su interpretación a código en lenguaje C y una correcta implementación de algoritmos para el proyecto.

En cuanto al trabajo en equipo, el aspecto más complejo fue la integración del proyecto, puesto que optamos por dividir el desarrollo en distintos módulos. Para lograrlo, las reuniones de coordinación virtuales fueron vitales para distribuir las tareas de cada integrante, y a su vez, brindar apoyo y resolver dudas en conjunto. Esta comunicación constante fue esencial para asegurar que las diferentes tareas fueran compatibles entre ellas y se completaran correctamente los objetivos del proyecto.